Table of Contents

Demo: API and Debugging

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

In this demo, we will set up and debug the backend API based on .NET 8.0. We will cover the initial setup of the project, configuring it for debugging, creating the API controller, using Data Transfer Objects (DTOs) to simplify the response data, and testing the API.

Setting Up Backend API

- **Create a New Project:** Begin by creating a new .NET project using the webapi template and name it api.
- **Update csproj File:** Copy the required PackageReference entries from your previous project's .csproj file into the new api.csproj. Ensure you include packages necessary for testing with Swagger UI.
- **Configure Program.cs:** Update Program.cs to set up the web application, including services like Swagger and CORS. Configure logging and routing, and adjust settings for development mode (e.g., database seeding and Swagger UI).
- **Update appsettings.json:** Add the ConnectionStrings section to appsettings.json for database connectivity.

Setting Up Debugging:

- **Create launch.json and task.json:** Add these files in the .vscode folder, ensuring the correct paths to your api and myshop folders are specified.
- **Run Debugging:** Access the API in the browser to see the Swagger UI.

Creating API Controller

- **Define API Controller:** Update ItemController.cs to use the [ApiController] attribute and configure routes for API endpoints. Implement actions like ItemList to handle HTTP GET requests.
- **Handle JSON Serialisation:** If you encounter issues with circular references in JSON responses, consider updating the Item model or configuring Newtonsoft.Json to handle reference loops.

Using Data Transfer Objects (DTOs)

- **Create DTOs:** Define DTOs to tailor data formats and simplify the API responses. Create a folder named DTOs and add DTO classes that exclude complex relationships like navigation properties.

Testing the Backend API

- Use Swagger
- Use terminal commands like curl
- Directly enter the API URL in the browser

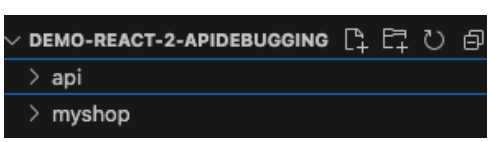
Setting up backend API

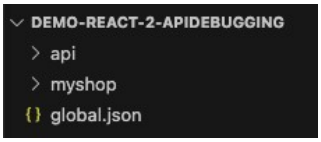
We start with creating a new dotnet project with the template webapi, and name it as api.

```
baifanz@Baifan-Mac Demo-react-2-apidebugging % dotnet new webapi -n api
```

In case you have many dotnet sdk installed, use `dotnet --list-sdks` to see all installed sdks. Use `dotnet new globaljson --sdk-version "version number"` to select the current sdk version.

After this, the folder structure looks as below (left) or (right):



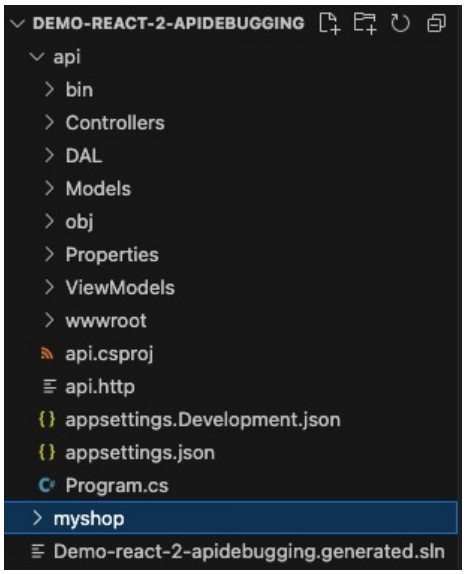


When converting your backend from MVC to a pure API, you'll copy the core files: **Controller**, **DAL**, and **Models**. You should not copy the **Views** folder, as an API's purpose is to serve data, not HTML pages.

For this project, we'll code new **API Controllers** directly alongside the old MVC controllers. This is a teaching method to help you compare the two architectures side-by-side, which is why the **ViewModels** folder and wwwroot are also included, even though it's not a standard practice for APIs.

In a professional setting, you would create a clean API project and use **DTOs (Data Transfer Objects)** instead of ViewModels to clearly define your data contracts and avoid mixing architectural patterns. In this demo, we keep both as a temporary solution so that you can easily compare the two architectures.

After copying, we get these:



After copying, check the **ItemDbContext.cs**, it should inherit from DbContext. If it is IdentityDbContext, change it to DbContext, because it only manages the Item data.

ItemDbContext.cs

api > DAL > ItemDbContext.cs > ItemDbContext

```
1  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
2  using Microsoft.EntityFrameworkCore;
3  using MyShop.Models;
4
5  namespace MyShop.DAL;
6
   11 references
7  public class ItemDbContext : DbContext
8  {
       0 references
9      public ItemDbContext(DbContextOptions<ItemDbContext> options) : base(options)
10     {
11         // Database.EnsureCreated();  // Remove this line if you use migrations
12     }
13
```

From the csproj file of your backend, copy the needed PackageReference into the new api.csproj. After copying, we get these:

api.csproj

api > api.csproj

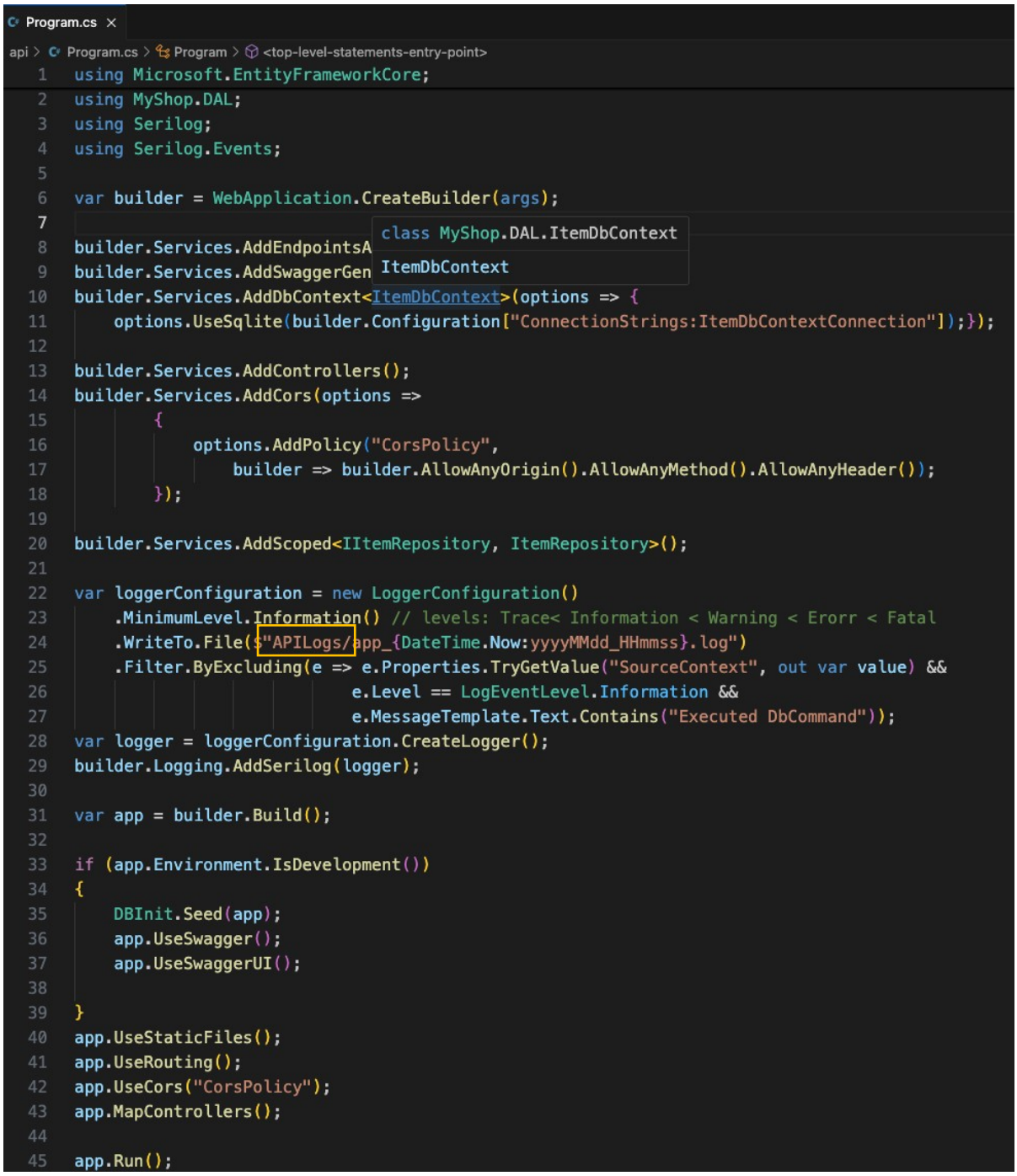
```
1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3    <PropertyGroup>
4      <TargetFramework>net8.0</TargetFramework>
5      <Nullable>enable</Nullable>
6      <ImplicitUsings>enable</ImplicitUsings>
7    </PropertyGroup>
8
9    <ItemGroup>
10     <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.7" />
11     <PackageReference Include="Swashbuckle.AspNetCore" Version="6.4.0" />
12     <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.8" />
13     <PackageReference Include="Microsoft.AspNetCore.Identity.UI" Version="8.0.8" />
14     <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.8">
15       <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
16       <PrivateAssets>all</PrivateAssets>
17     </PackageReference>
18     <PackageReference Include="Microsoft.EntityFrameworkCore.Proxies" Version="8.0.7" />
19     <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.7" />
20     <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.8">
21       <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
22       <PrivateAssets>all</PrivateAssets>
23     </PackageReference>
24     <PackageReference Include="Serilog.Extensions.Logging" Version="8.0.0" />
25     <PackageReference Include="Serilog.Sinks.File" Version="6.0.0" />
26   </ItemGroup>
27
28 </Project>
```

Line10-11: these packages are necessary for testing the backend api with Swagger UI.

You can notice that we dropped `Microsoft.VisualStudio.Web.CodeGeneration.Design` because it is a design-time tool used for scaffolding code (like controllers and API endpoints), and it is not a runtime dependency needed for the API to function.

Optional: You can go to the terminal, inside the api folder (cd api), run `dotnet restore` to install these packages.
We can also directly install things in running the projects later.

Modify the Program.cs to this:




```
1  using Microsoft.EntityFrameworkCore;
2  using MyShop.DAL;
3  using Serilog;
4  using Serilog.Events;
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  builder.Services.AddEndpointsA
9  builder.Services.AddSwaggerGen
10 builder.Services.AddDbContext<ItemDbContext>(options => {
11     options.UseSqlite(builder.Configuration["ConnectionStrings:ItemDbContextConnection"]);});
12
13 builder.Services.AddControllers();
14 builder.Services.AddCors(options =>
15         {
16             options.AddPolicy("CorsPolicy",
17                 builder => builder.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
18         });
19
20 builder.Services.AddScoped<IItemRepository, ItemRepository>();
21
22 var loggerConfiguration = new LoggerConfiguration()
23     .MinimumLevel.Information() // levels: Trace< Information < Warning < Erorr < Fatal
24     .WriteTo.File($"APILogs/app_{DateTime.Now:yyyyMMdd_HHmmss}.log")
25     .Filter.ByExcluding(e => e.Properties.TryGetValue("SourceContext", out var value) &&
26                              e.Level == LogEventLevel.Information &&
27                              e.MessageTemplate.Text.Contains("Executed DbCommand"));
28 var logger = loggerConfiguration.CreateLogger();
29 builder.Logging.AddSerilog(logger);
30
31 var app = builder.Build();
32
33 if (app.Environment.IsDevelopment())
34 {
35     DBInit.Seed(app);
36     app.UseSwagger();
37     app.UseSwaggerUI();
38
39 }
40 app.UseStaticFiles();
41 app.UseRouting();
42 app.UseCors("CorsPolicy");
43 app.MapControllers();
44
45 app.Run();
```

Explanations:
Line6: Creates a builder for configuring and setting up the web application. It handles services, configurations, and middleware.
AddEndpointsApiExplorer: Enables API endpoint discovery for Swagger.
AddSwaggerGen: Adds Swagger generation services, allowing automatic documentation of API endpoints.
AddCors: Configures CORS (Cross-Origin Resource Sharing), allowing requests from any origin, with any method and any header, for convenient debugging and testing.
Line22-Line29: configure and add the logger. Use APILogs as the log folder for clarity.
Line33-Line39, In development mode:

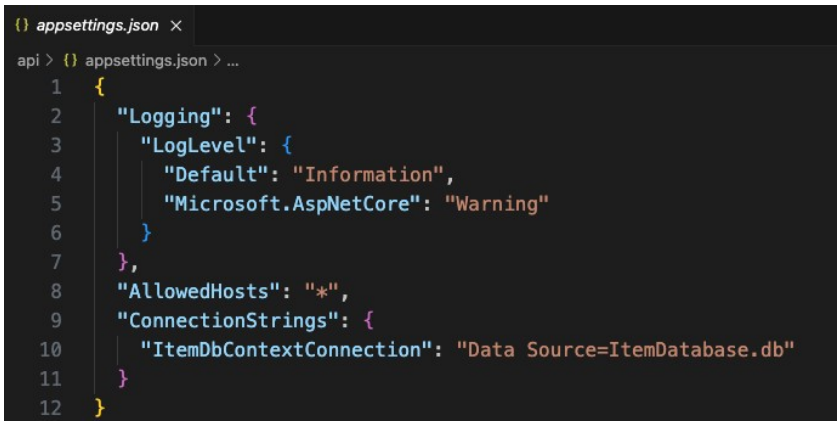
- **DBInit.Seed**: Seeds the database with initial data.
- **UseSwagger**: Enables the Swagger endpoint.
- **UseSwaggerUI**: Enables the Swagger UI for API testing and documentation.

app.UseStaticFiles: Allow using files in wwwroot.
app.UseRouting: Adds routing middleware to the pipeline.
app.UseCors: Applies the defined CORS policy to incoming requests
MapControllers: enables attribute routing for both MVC and API controllers, which is the modern and professional way to define API endpoints. It maps requests to controller actions based on the [Route] attributes you explicitly place on your classes and methods, giving you full control over the URL structure. This is in contrast to conventional routing which relies on a predefined global pattern.

In the next, we want to run the backend in development mode, because the DBseeding and Swagger UI are both only available in development code in the Program.cs.
One way is to configure your environment variable ASPNETCORE_ENVIRONMENT, and them run the project (You could also simply remove the condition in Line 33 so that the DBseeding and Swagger UI are always available.)

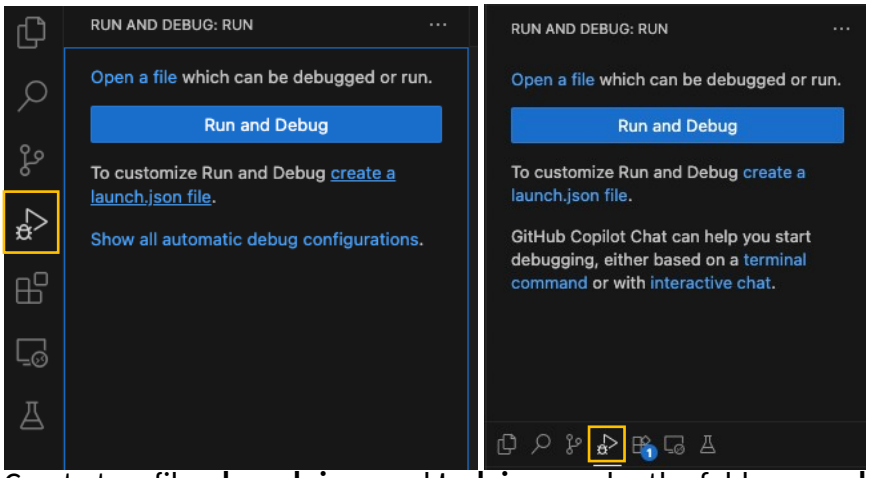
We use another way, that is simply to run it in debug mode. We need to set up the debugging environment in the next section.

Also remember to change the appsettings.json to add the ConnectionStrings:



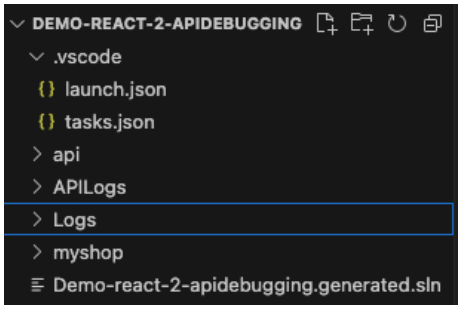
Setting up debugging of the backend API

We go to the debug panel and click create a launch.json file:

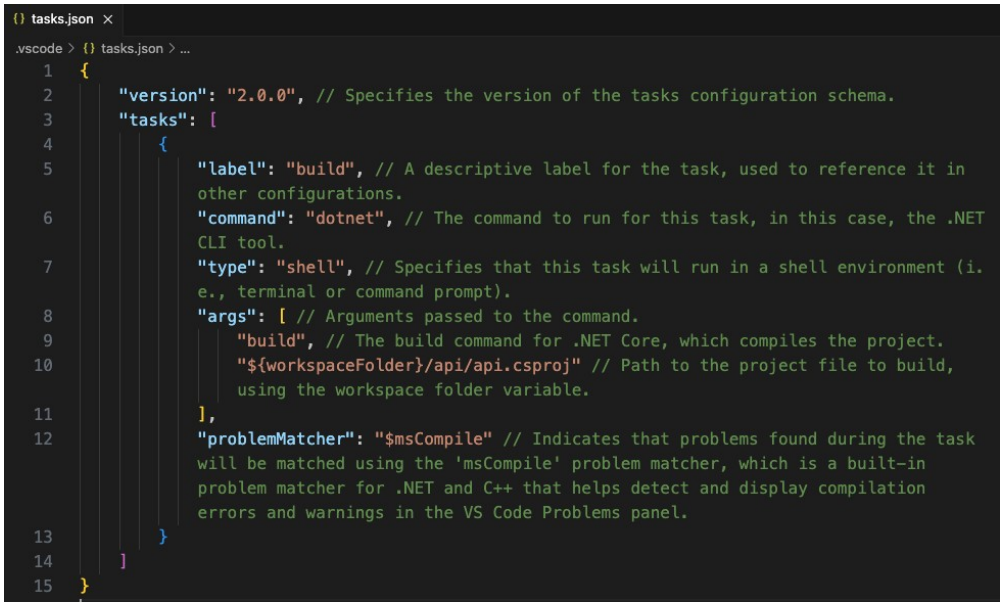


Create two files, **launch.json** and **task.json** under the folder **.vscode**.

When creating **launch.json**, you could click `create a launch.json file` , then select `C#`, then click `Add Configuration` (at right bottom corner, it normally auto pops up), then `.NET: Launch Executable File (Web)`.
We run the debugging directly in the parental folder of **api** and **myshop**, so the folder and files look like:



The two files look like (pay special attention to the path, api is the backend folder):

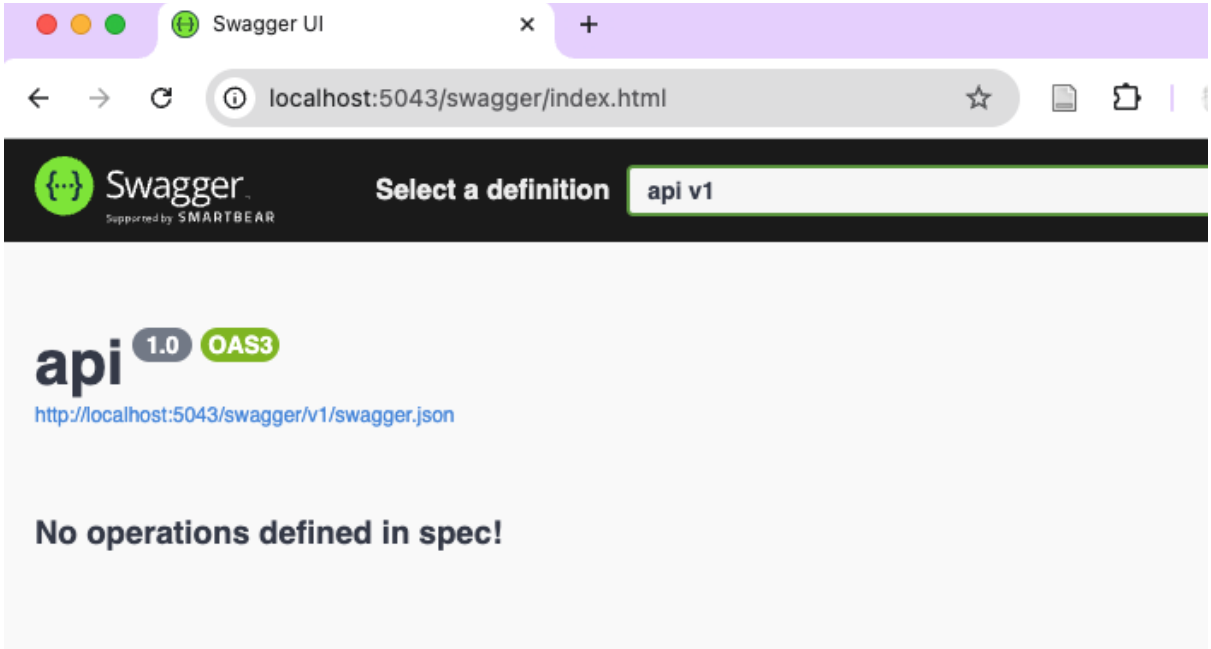


launch.json

.vscode > launch.json > ...

```jsonc
{
    // Use IntelliSense to learn about possible attributes.
    // Hover to view descriptions of existing attributes.
    // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
    "version": "0.2.0",
    "configurations": [
    {
        "name": ".NET Core Launch (web)", // Name of the launch configuration as shown in the debug panel
        "type": "coreclr", // Type of the debugger to use, in this case, .NET Core CLR (Common Language Runtime)
        "request": "launch", // Request type, here it is set to launch a new process
        "preLaunchTask": "build", // Task to run before launching the debugger, typically to build the project
        "program": "${workspaceFolder}/api/bin/Debug/net8.0/api.dll", // Path to the compiled .NET Core application (the entry point for the application)
        "args": [],
        "cwd": "${workspaceFolder}/api", // Current working directory
        "stopAtEntry": false,
        "serverReadyAction": {
            "action": "openExternally",
            "pattern": "\\bNow listening on:\\s+(http?://\\S+)",
            "uriFormat": "%s/swagger"
        },
        "env": { // Environment variables to set when launching the application
            "ASPNETCORE_ENVIRONMENT": "Development"
        },
    }
    ]
}
```

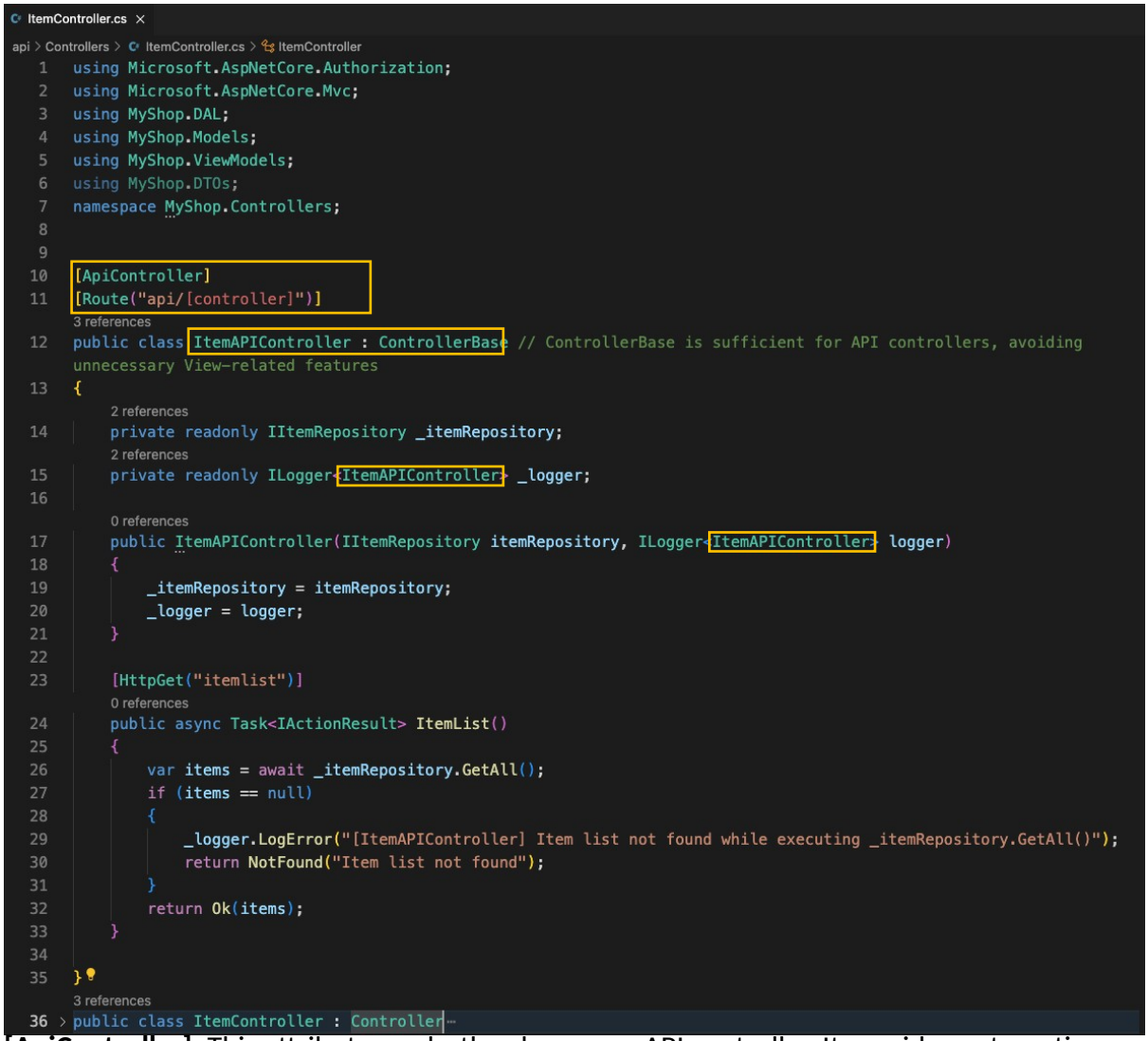
Now run the project in debugging mode by pressing F5.
It should automatically open the browser and go to http://<your-url>/swagger/index.html. If not, you could open the browser, go to the address:
http://localhost:5043/swagger/index.html (replace it with your URL in the api/Properties/launchSettings.json)
You will see the following results:



The reason is that the current controller is defined for MVC, not for API. In the next section we define the controller for API.

Creating API controller

In the ItemController.cs, add the line10-line 35.



[ApiController]: This attribute marks the class as an API controller. It provides automatic model validation, error handling, and binding for API requests.

[Route("api/[controller]")]: This attribute defines the base route for all actions in this controller. [controller] is a placeholder that will be replaced with the name of the controller (minus the "Controller" suffix). For this controller named ItemAPIController, the base route will be api/itemapi.

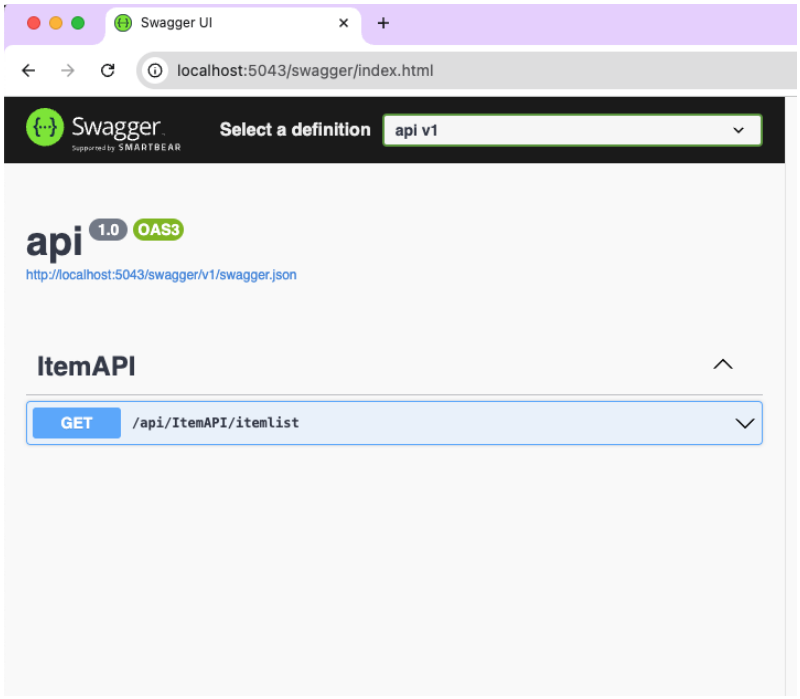
[HttpGet("itemlist")]: This attribute specifies that the ItemList method should handle HTTP GET requests at the route api/itemapi/itemlist. It maps the GET request to this method.

public async Task<IActionResult> ItemList(): This is an asynchronous method that returns an IActionResult. The async keyword indicates that the method will perform asynchronous operations.

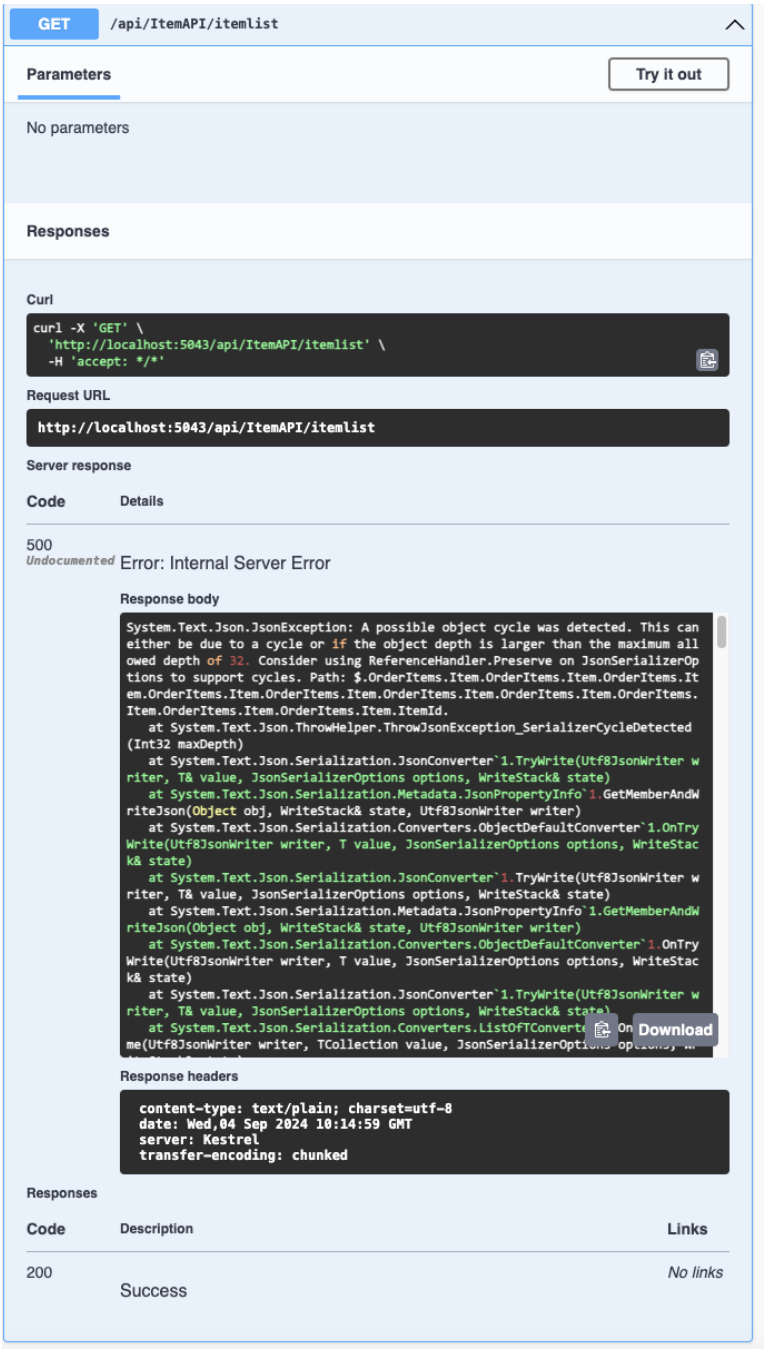
if (items == null): Checks if the retrieved items are null. If so, it logs an error message and returns a 404 Not Found response with the message "Item list not found".

return Ok(items);: If the items are successfully retrieved and not null, it returns a 200 OK response with the items as the response body.

Now we run the api by pressing F5, and see one operation is defined:


Swagger UI
localhost:5043/swagger/index.html
Swagger
Supported by SMARTBEAR
Select a definition
api v1
api
1.0
OAS3
http://localhost:5043/swagger/v1/swagger.json
ItemAPI
GET
/api/ItemAPI/itemlist

But if you try to execute it, you will see a success return with an error message.



The reason is that it encountered a cycle problem of the returned json file, which originates from the infinite cross references between the *OrderItems.Item.OrderItems.Item.OrderItems…* We take a look at the the **Item.cs**, then you can see the Item refers the **OrderItems**, and **OrderItems** also refer to **Items**, forming an infinite loop.

Item.cs

```
api > Models > Item.cs > ...
1   using System.ComponentModel.DataAnnotations;
2
3   namespace MyShop.Models
4   {
        23 references
5       public class Item
6       {
            2 references
7           public int ItemId { get; set; }
8
9           [RegularExpression(@"[0-9a-zA-ZæøåÆØÅ. \-]{2,20}", ErrorMessage = "The Name
            must be numbers or letters and between 2 to 20 characters.")]
10          [Display(Name = "Item name")]
            9 references
11          public string Name { get; set; } = string.Empty;
12
13          [Range(0.01, double.MaxValue, ErrorMessage = "The Price must be greater than
            0.")]
            10 references
14          public decimal Price { get; set; }
15
16          [StringLength(200)]
            8 references
17          public string? Description { get; set; }
18
            8 references
19          public string? ImageUrl { get; set; }
20          // navigation property
            0 references
21          public virtual List<OrderItem>? OrderItems { get; set; }
22      }
23  }
```

OrderItem.cs

```
api > Models > OrderItem.cs > ...
1   namespace MyShop.Models;
2
    11 references
3   public class OrderItem
4   {
        0 references
5       public int OrderItemId { get; set; }
        7 references
6       public int ItemId { get; set; }
7       //navigation property
        2 references
8       public virtual Item Item { get; set; } = default!;
        7 references
9       public int Quantity { get; set; }
        6 references
10      public int OrderId { get; set; }
11      //navigation property
        1 reference
12      public virtual Order Order { get; set; } = default!;
        3 references
13      public decimal OrderItemPrice { get; set; }
14  }
```

Avoiding infinite cross-reference

To prevent this, a simple way is to comment out the line21 in Item.cs:

```
20          // navigation property
21          // public virtual List<OrderItem>? OrderItems { get; set; }
```

We also introduce another way that stops infinite looping of the returned json.
Go to the terminal, in the api folder, run the below command. We specify the version to avoid installing the latest version that is only compatible for .net 9

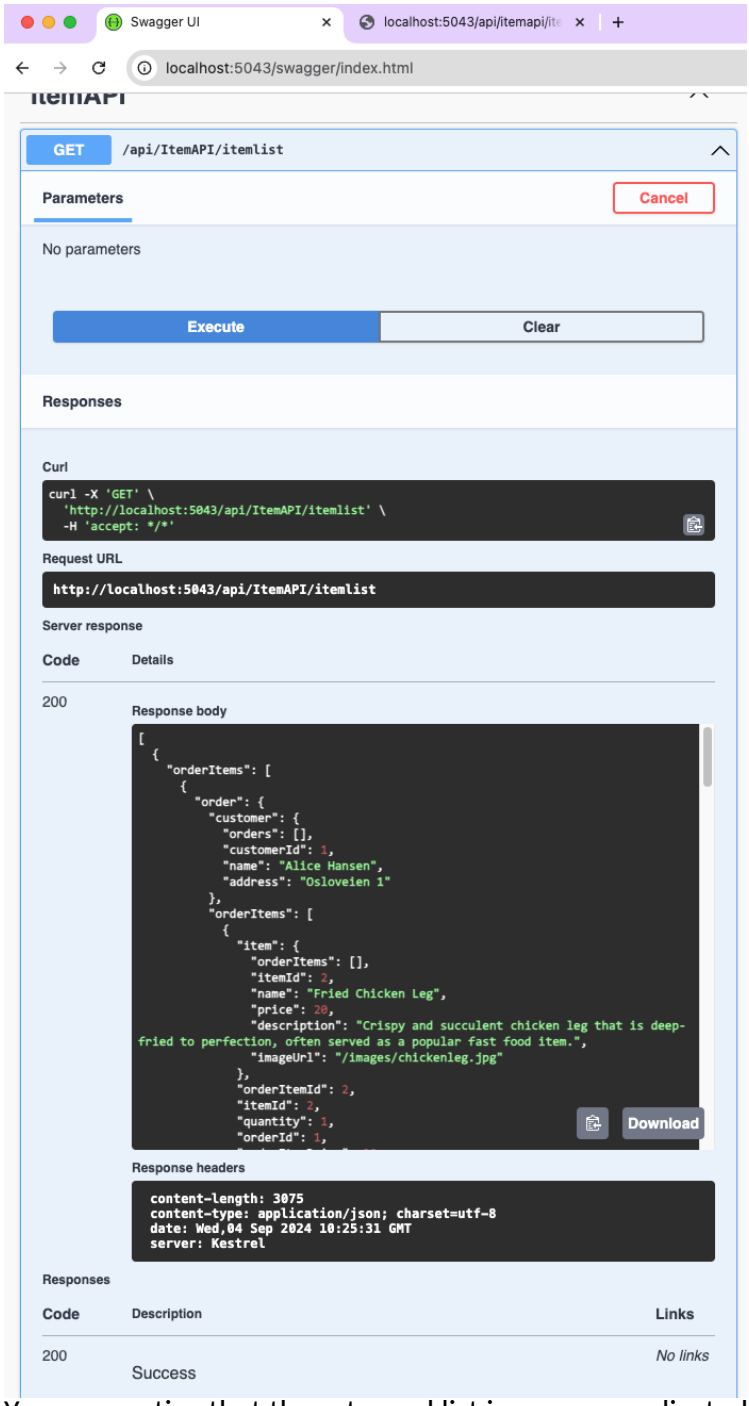
```
dotnet add package Microsoft.AspNetCore.Mvc.NewtonsoftJson --version "8.*"
```

In Program.cs, change the **builder.Services.AddControllers()** to add the service (the order of serves does not matter).

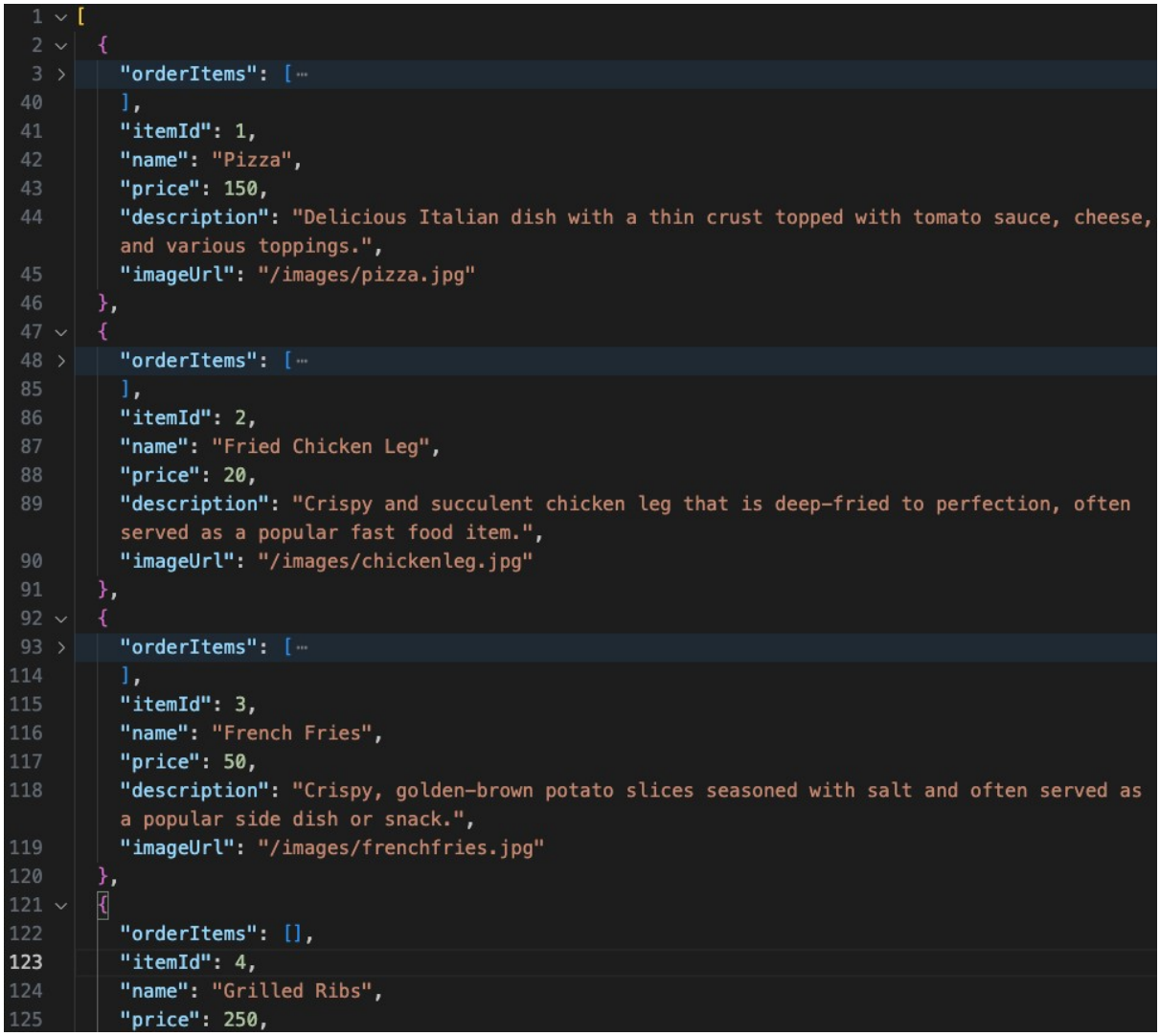
```
8   builder.Services.AddControllers().AddNewtonsoftJson(options =>
9   {
10      options.SerializerSettings.ReferenceLoopHandling = Newtonsoft.Json.ReferenceLoopHandling.Ignore;
11  });
```

Now restart the debugging mode with the button:

Now the get method should return the correct results:



You may notice that the returned list is more complicated than a simple item list. If we copy the results out, and collapse the orderItems, then we can find it is indeed an itemlist but some of them have the orderItems list as a property. The reason is that the Item.cs model has a navigation property in line 21.



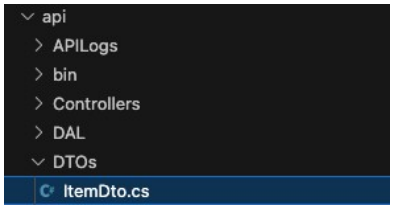
A common practice is to create extra data models for data transferred to the client. We do this in the next section.

Data Transfer Object

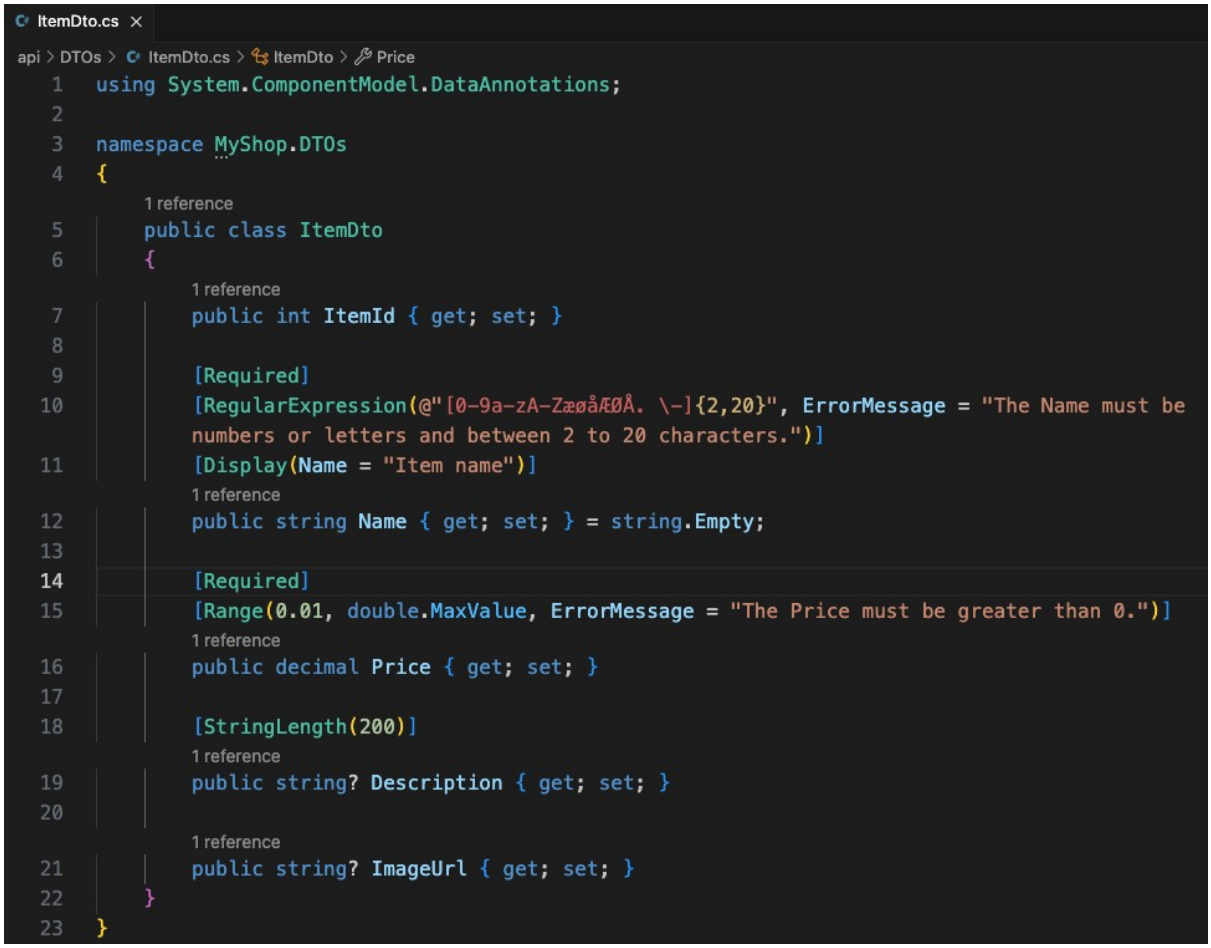
Using Data Transfer Objects (DTOs) is generally recommended for most applications due to the benefits of:

- separation of concerns: DTOs help separate the data structure used by the API from the internal domain model. This makes your API more adaptable to changes in the internal domain model and vice versa.
- security: DTOs allow you to control and limit what data is exposed to clients
- flexibility: DTOs allow you to tailor the data format and structure to meet the needs of the client and API consumers.
- better control over data handling: DTOs can include validation logic that ensures data integrity before it is processed by your application.

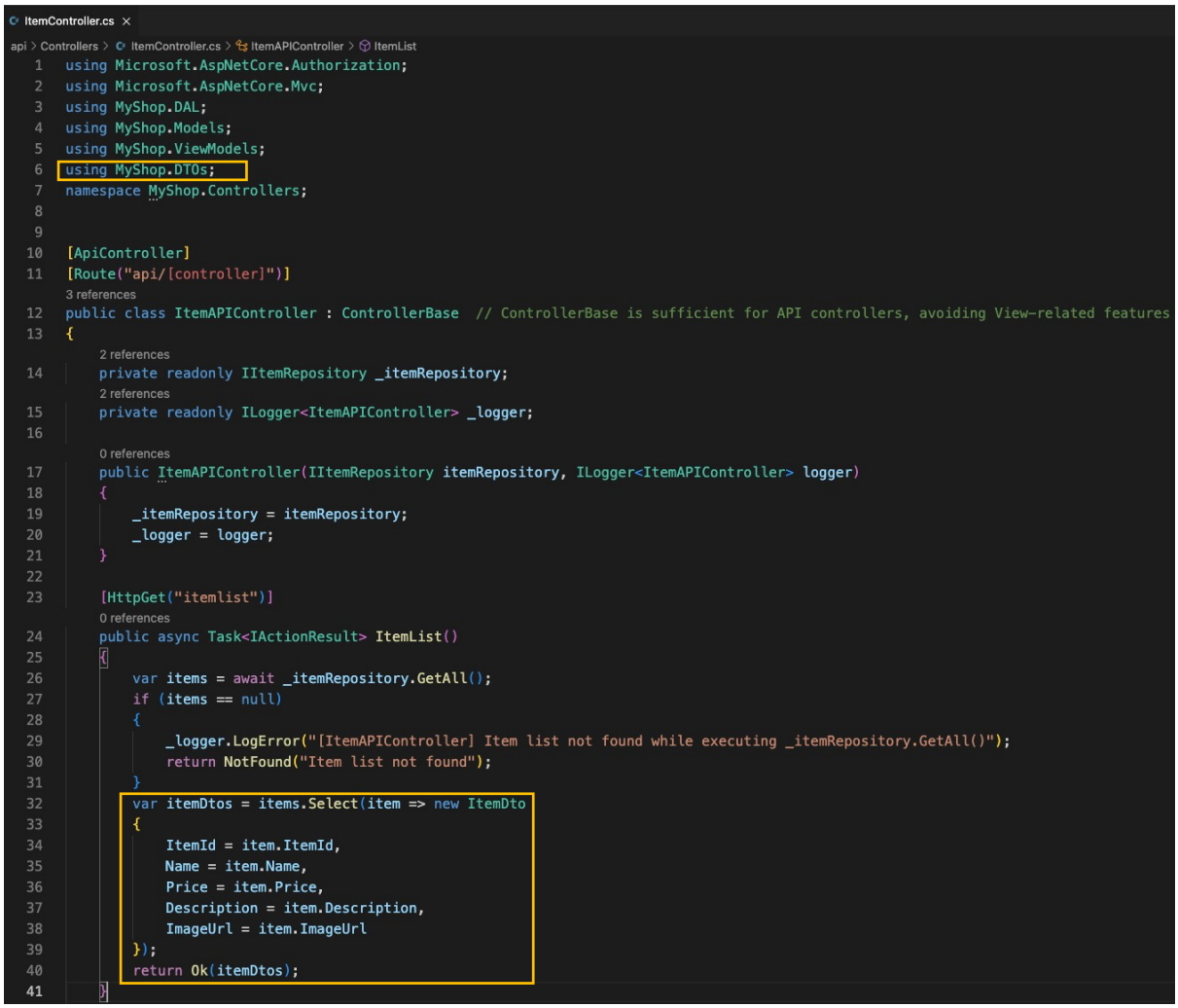
We create a folder named as DTOs under the api folder



We create a file ItemDto.cs under DTOs. It is essential the same as the Item.cs, but without the navigation property.

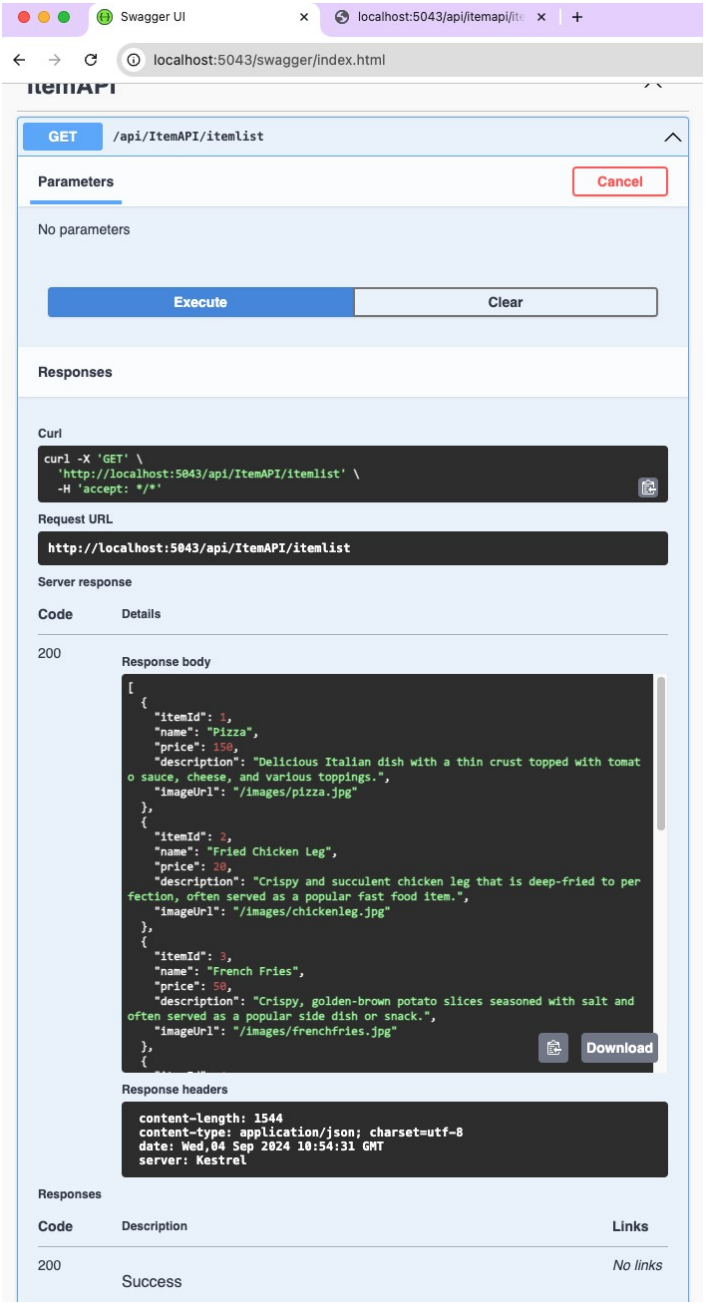


Then we change the HttpGet method slightly (remember using MyShop.DTOs):



```
ItemController.cs
api > Controllers > ItemController.cs > ItemAPIController > ItemList
1   using Microsoft.AspNetCore.Authorization;
2   using Microsoft.AspNetCore.Mvc;
3   using MyShop.DAL;
4   using MyShop.Models;
5   using MyShop.ViewModels;
6   using MyShop.DTOs;
7   namespace MyShop.Controllers;
8
9
10  [ApiController]
11  [Route("api/[controller]")]
    3 references
12  public class ItemAPIController : ControllerBase  // ControllerBase is sufficient for API controllers, avoiding View-related features
13  {
        2 references
14      private readonly IItemRepository _itemRepository;
        2 references
15      private readonly ILogger<ItemAPIController> _logger;
16
        0 references
17      public ItemAPIController(IItemRepository itemRepository, ILogger<ItemAPIController> logger)
18      {
19          _itemRepository = itemRepository;
20          _logger = logger;
21      }
22
23      [HttpGet("itemlist")]
        0 references
24      public async Task<IActionResult> ItemList()
25      {
26          var items = await _itemRepository.GetAll();
27          if (items == null)
28          {
29              _logger.LogError("[ItemAPIController] Item list not found while executing _itemRepository.GetAll()");
30              return NotFound("Item list not found");
31          }
32          var itemDtos = items.Select(item => new ItemDto
33          {
34              ItemId = item.ItemId,
35              Name = item.Name,
36              Price = item.Price,
37              Description = item.Description,
38              ImageUrl = item.ImageUrl
39          });
40          return Ok(itemDtos);
41      }
```

Now each time, an itemlist is received from the database, it is mapped to the ItemDto. Now restart debugging, you will see the returned itemlist much simpler:

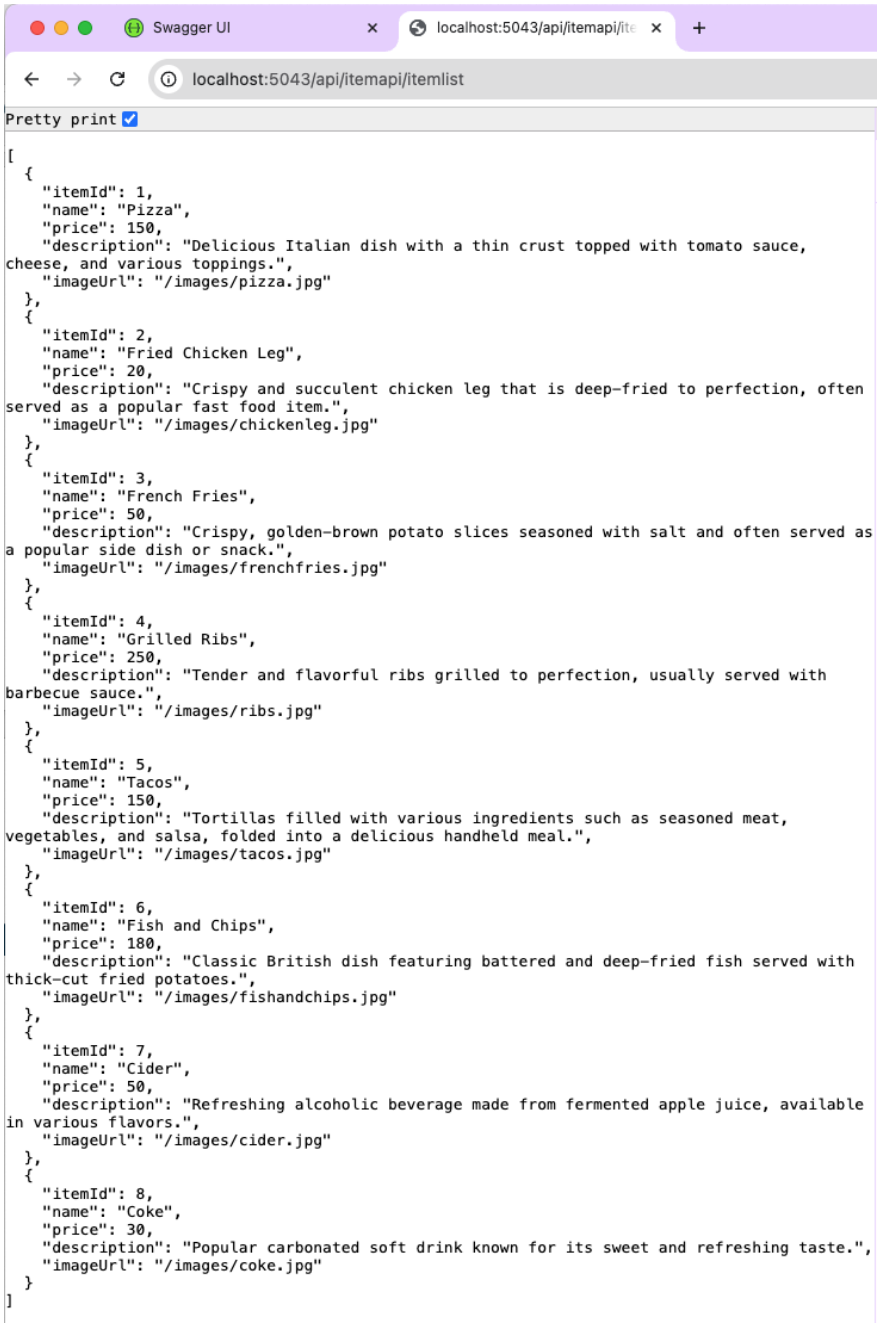


Alternative ways of testing the backend API

There are other ways to test the backend without using Swagger.
In the terminal, using the curl command:
curl http://localhost:5043/api/itemapi/itemlist

```
(py38) baifanz@Baifan-Mac api % curl http://localhost:5043/api/itemapi/itemlist
[{"itemId":1,"name":"Pizza","price":150.0,"description":"Delicious Italian dish with a thin crust topped with tomato sauce, cheese
, and various toppings.","imageUrl":"/images/pizza.jpg"},{"itemId":2,"name":"Fried Chicken Leg","price":20.0,"description":"Crispy
 and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.","imageUrl":"/images/chicke
nleg.jpg"},{"itemId":3,"name":"French Fries","price":50.0,"description":"Crispy, golden-brown potato slices seasoned with salt and
 often served as a popular side dish or snack.","imageUrl":"/images/frenchfries.jpg"},{"itemId":4,"name":"Grilled Ribs","price":25
0.0,"description":"Tender and flavorful ribs grilled to perfection, usually served with barbecue sauce.","imageUrl":"/images/ribs.
jpg"},{"itemId":5,"name":"Tacos","price":150.0,"description":"Tortillas filled with various ingredients such as seasoned meat, veg
etables, and salsa, folded into a delicious handheld meal.","imageUrl":"/images/tacos.jpg"},{"itemId":6,"name":"Fish and Chips","p
rice":180.0,"description":"Classic British dish featuring battered and deep-fried fish served with thick-cut fried potatoes.","ima
geUrl":"/images/fishandchips.jpg"},{"itemId":7,"name":"Cider","price":50.0,"description":"Refreshing alcoholic beverage made from
fermented apple juice, available in various flavors.","imageUrl":"/images/cider.jpg"},{"itemId":8,"name":"Coke","price":30.0,"desc
ription":"Popular carbonated soft drink known for its sweet and refreshing taste.","imageUrl":"/images/coke.jpg"}]
```

Or in the browser, simply put the api URL in the address bar:




```
[
  {
    "itemId": 1,
    "name": "Pizza",
    "price": 150,
    "description": "Delicious Italian dish with a thin crust topped with tomato sauce,
cheese, and various toppings.",
    "imageUrl": "/images/pizza.jpg"
  },
  {
    "itemId": 2,
    "name": "Fried Chicken Leg",
    "price": 20,
    "description": "Crispy and succulent chicken leg that is deep-fried to perfection, often
served as a popular fast food item.",
    "imageUrl": "/images/chickenleg.jpg"
  },
  {
    "itemId": 3,
    "name": "French Fries",
    "price": 50,
    "description": "Crispy, golden-brown potato slices seasoned with salt and often served as
a popular side dish or snack.",
    "imageUrl": "/images/frenchfries.jpg"
  },
  {
    "itemId": 4,
    "name": "Grilled Ribs",
    "price": 250,
    "description": "Tender and flavorful ribs grilled to perfection, usually served with
barbecue sauce.",
    "imageUrl": "/images/ribs.jpg"
  },
  {
    "itemId": 5,
    "name": "Tacos",
    "price": 150,
    "description": "Tortillas filled with various ingredients such as seasoned meat,
vegetables, and salsa, folded into a delicious handheld meal.",
    "imageUrl": "/images/tacos.jpg"
  },
  {
    "itemId": 6,
    "name": "Fish and Chips",
    "price": 180,
    "description": "Classic British dish featuring battered and deep-fried fish served with
thick-cut fried potatoes.",
    "imageUrl": "/images/fishandchips.jpg"
  },
  {
    "itemId": 7,
    "name": "Cider",
    "price": 50,
    "description": "Refreshing alcoholic beverage made from fermented apple juice, available
in various flavors.",
    "imageUrl": "/images/cider.jpg"
  },
  {
    "itemId": 8,
    "name": "Coke",
    "price": 30,
    "description": "Popular carbonated soft drink known for its sweet and refreshing taste.",
    "imageUrl": "/images/coke.jpg"
  }
]
```


Congratulations! Now you have a running API and learnt how to test it! ☺